APPLIED MACHINE LEARNING CASE STUDY

AI Support Ticket Triage and Quality Review System

Keisha Walker

An evaluated assistive workflow for banking support messages

The prototype suggests a support queue and intent, extracts a short source-based summary, and surfaces reasons for human review. Its strongest result is a reproducible intent-classification comparison, paired with explicit evidence of where the surrounding workflow still fails.

Scope matters. This is a limited demonstration using the public BANKING77 English banking-intent benchmark. It is not an IT-ticket dataset, its production origin is not verified here, and the app performs no account or financial actions.

Results at a glance

Measure	Observed result	What it establishes
77-intent test macro-F1	0.903	Higher than the 0.877 baseline on this fixed benchmark
Full review policy	69.87% coverage	5.34% intent error among unflagged messages
Summary phrase retention	82 of 99 phrases	Lexical diagnostic only; all phrases retained in 19 of 36 cases
Software checks	26 tests passed	Checks implementation behavior, not real-world service quality

Business problem and design response

A support worker must understand the issue, choose an appropriate destination and preserve details for the next person. Misrouting, missing context and overconfident recommendations are plausible risks. The project makes those decisions inspectable without claiming that it has reduced handling time or improved customer outcomes.

The interface keeps the original message beside the suggested intent, estimated confidence, alternatives, extractive summary and review reasons. A person owns every routing and resolution decision, including cases where no review rule fires.

Main conclusion

The richer word-and-character model is the better candidate in this experiment. The complete workflow remains a supervised demonstration: review heuristics reduce coverage without improving accepted-case error, and small diagnostic sets reveal missed off-domain and ambiguous requests. Production readiness would require independently reviewed target-domain data and operational evaluation.

Evidence: frozen experiment outputs generated on 18 October 2026. References and reproduction instructions appear on page 8.

02 CASE STUDY

The assistive workflow

The system is intentionally narrow: a local, single-message suggestion tool rather than an autonomous agent or an integrated service desk.

Stage	Implementation	Human responsibility
1 Enter a message	Public or synthetic English banking-support text; 5,000-character cap	Remove private data before use
2 Estimate the intent	77-class model; top three alternatives; probability for the top intent	Check what the message is actually asking
3 Suggest a queue	Fixed mapping from 77 intents to nine project-defined queues	Confirm the queue and any specialist need
4 Extract a summary	Rank and copy up to two sentences; cap at 60 words	Compare with the full source and missing context
5 Inspect review reasons	Confidence, margin, domain, context and topic rules	Make the final routing and response decision

What the interface makes visible

The Streamlit app displays the suggested category, fine-grained intent, confidence, top alternatives and a review flag. Confidence means the model's estimated probability for the 77-class intent; it is not the probability that the broader queue is correct and is not a correctness guarantee. Users can download the result as JSON.

A review flag is an explanation prompt, not a separate learned quality score. The checklist asks whether the queue fits the issue, whether the summary preserves negation and timing, whether multiple issues or a security concern appear, and whether a specialist should inspect the original.

A transparent operational taxonomy

The nine queues are Card lifecycle; Card payments; Top ups and deposits; Transfers; Cash and ATMs; Account access and profile; Identity verification; Foreign exchange and coverage; and Security and disputes. This mapping is defined in project code before evaluation. BANKING77 does not provide queue labels, routing outcomes or an organization's ownership model.

Privacy and action boundaries

The model runs in the local Python process. The application contains no model API calls, account access, automatic ticket routing or ticket persistence. Hosting the app elsewhere changes who can receive input, so the local-only assumption must be revisited before deployment.

Basic masking catches email-like strings and long number sequences. It is not comprehensive de-identification or a data-loss-prevention control. Do not enter passwords, card numbers, identity documents or private customer records. The tool must not be used for credit, fraud adjudication, eligibility or account changes.

Why this matters. An inspectable suggestion can support an operator, but a confident prediction or an unflagged message is never permission to act.

03 CASE STUDY

Benchmark and evaluation design

The experiment separates model fitting, model selection and final reporting, while documenting the limits of the benchmark itself.

Data provenance and permitted claims

BANKING77 by PolyAI contains 10,003 official training examples, 3,080 official test examples and 77 banking intents. The official test has 40 rows per intent. The source describes online-banking queries; the collection method and whether all queries originated in real customer interactions are not established here. It should not be called verified production ticket data or definitively synthetic data. [1-3]

There are no source labels for urgency, priority, service-level agreements, handling time, resolution quality or customer satisfaction. The nine queues and review rules are project additions, not supervised ground truth for those business outcomes.

Exact duplicate hygiene

Development data step	Rows removed or retained
Original official development split	10,003 rows
Remove normalized text overlapping official test	25 removed
Remove conflicting-label normalized strings	2 removed
Remove remaining normalized development duplicates	30 removed
Stratified split of remaining development data	7,956 train / 1,990 validation
Official test rows, order and labels	3,080 retained unchanged

Normalization lowercases text, replaces non-word runs with a space and strips edges. Official test text is used only for this disclosed exact-overlap hygiene, which removes development rows rather than altering the test set. No validation or test text fits a vocabulary or classifier.

Residual similarity remains. Four normalized duplicate test rows are retained. A post-fit audit finds 40 test rows with word-TF-IDF cosine similarity of at least 0.95 to a training row. These observations limit independence claims; similar paraphrases can survive exact deduplication.

Selection and reporting discipline

The development split is stratified by intent with seed 42. The model is selected on validation intent macro-F1; the review threshold is selected on validation coverage subject to an empirical error target. The official test is then used for final reporting. No model, threshold, summary rule or review rule is retuned after seeing test scores or diagnostic outcomes.

Source records: data/DATA_PROVENANCE.md, data/raw/SOURCE_MANIFEST.json and reports/split_audit.json. The preliminary provenance audit used a simpler normalization; the table above reports the final implemented policy.

04 CASE STUDY

Model comparison and calibration

The comparison tests a simple, interpretable baseline against a richer sparse-text representation. It does not establish that one model family is inherently superior.

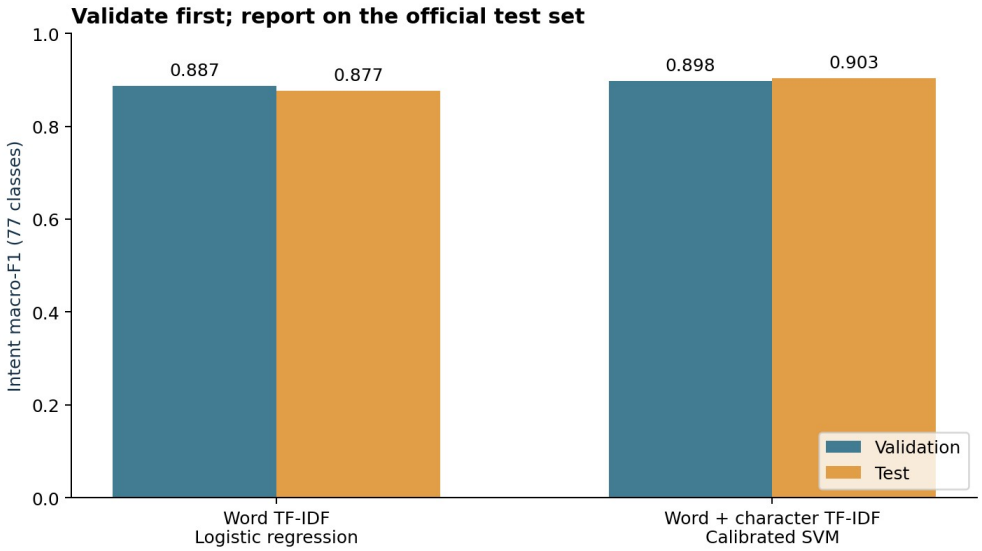


Figure 1. Model selection uses validation macro-F1; test results are reported after selection.

Metric	Word logistic baseline	Word and character SVM
Validation intent macro-F1	0.886712	0.898048
Test intent macro-F1	0.876571	0.903257
Test nine-queue macro-F1	0.946403	0.956293
Test log loss	0.766	0.400
Test calibration error in 10 bins	0.264	0.058

What changed between models

The baseline uses word TF-IDF unigrams and bigrams with logistic regression (C=4). The candidate combines the same word features with character-within-word 3-5-grams and a linear SVM (C=1). Sigmoid calibration uses three stratified training-only folds; the full vectorizer-and-classifier pipeline is cross-fitted to avoid leaking calibration-fold vocabulary.

Character fragments provide a plausible way to represent spelling variation, but the experiment changes features, estimator and calibration together. It cannot isolate which change causes the observed gain. The 0.0267 test macro-F1 increase is evidence for this candidate configuration on this benchmark.

The 0.956 queue macro-F1 is a separate, easier aggregation: several intent errors remain inside the same broad queue. It must not replace the 77-intent score. Lower calibration error and log loss are encouraging within-domain, but do not validate confidence on unseen domains.

Uncertainty: a 500-replicate paired row bootstrap gives a 95% percentile interval of 0.892-0.912 for candidate test macro-F1 and 0.0185-0.0353 for its improvement. These intervals omit training variability, domain shift and correlated paraphrases.

05 CASE STUDY

Selective review and actual coverage

The app combines a confidence threshold with additional fixed review rules. The full policy must be evaluated separately from the confidence-only curve.

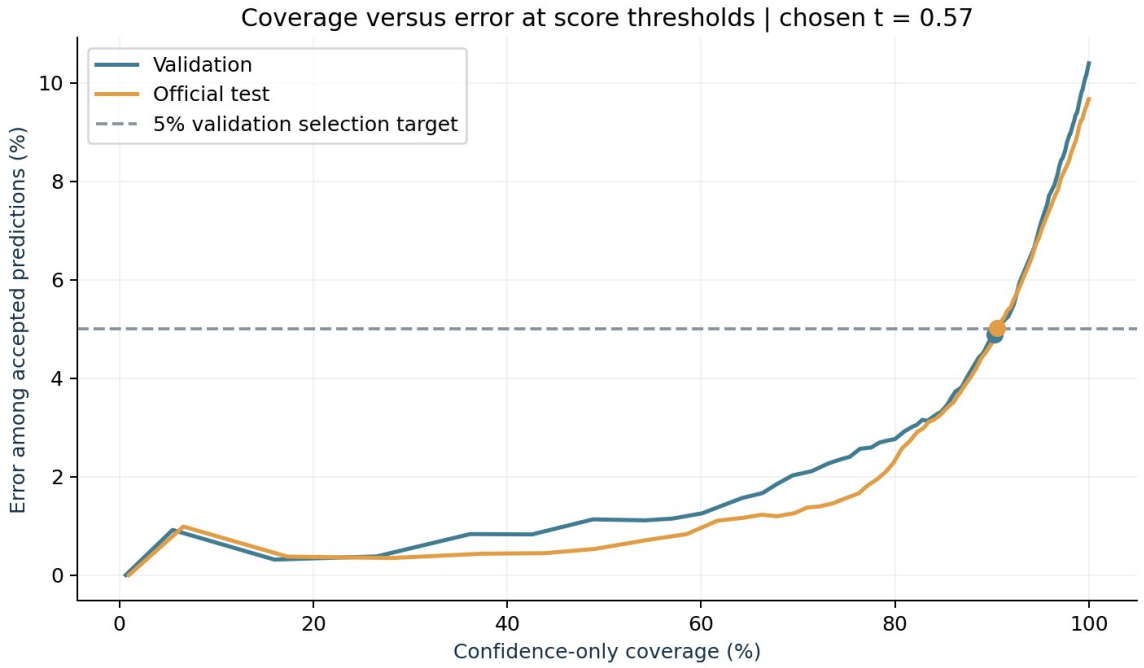


Figure 2. Each curve varies only the confidence threshold. It does not represent the full app policy.

Policy and split	Accepted	Coverage	Intent error among accepted
Confidence only on validation	1,797 / 1,990	90.30%	88 / 1,797 = 4.90%
Confidence only on test	2,788 / 3,080	90.52%	140 / 2,788 = 5.02%
Full app policy on test	2,152 / 3,080	69.87%	115 / 2,152 = 5.34%

How the operating point is selected

Threshold 0.57 maximizes validation coverage among 0.01-grid thresholds with empirical accepted-case error at or below 5% and at least 100 accepted examples. Ties use the lower threshold; if none qualifies, the fallback is review all. The 5% target is a selection criterion, not a guaranteed test or production error bound.

The full policy also flags a top-two intent margin below 0.10, weak banking cues, fewer than two known terms, fewer than four words, possible multiple issues, security or dispute topics, and summary truncation. “Accepted” means no rule fired, not that the system autonomously acted.

Important negative result. The full policy reviews 928 of 3,080 test messages and leaves 115 errors among 2,152 unflagged messages. Its unflagged subset contains fewer total errors, but the accepted-case error rate is higher than confidence-only filtering. These heuristics do not demonstrate improved selective error on this test set.

95% Wilson intervals for test accepted-case error: confidence-only 4.27%-5.90%; full policy 4.47%-6.38%. Neither interval is a guarantee under domain shift. The operating policy remains unchanged after this evaluation.

06 CASE STUDY

Summary and review diagnostics

Project-authored synthetic fixtures expose failure modes without pretending to be human-rated, externally validated or representative customer data.

Extractive summary method

The summarizer ranks sentences using issue cues, banking-domain cues and a bounded length term, then copies at most two sentences in source order. It caps output at 60 whitespace-separated words and marks truncation. One-sentence inputs may be unchanged. Sentence splitting is rule-based and can mishandle abbreviations.

Thirty-six independently authored fixture messages contain 99 expected key phrases. Each phrase is a case-insensitive contiguous source span. Expectations were frozen before inference and were not supplied by human raters. The comparator simply copies the first two sentences.

Summary diagnostic	Lead two sentences	Issue-focused extractor
Expected phrases retained by micro average	51 / 99 = 51.52%	82 / 99 = 82.83%
All expected phrases retained	7 / 36 = 19.44%	19 / 36 = 52.78%
Summaries fully copied from source	36 / 36	36 / 36
Mean summary / input word count	51.73%	57.58%

The issue-focused method retains more authored phrases, partly at greater output length. It still misses at least one expected phrase in 17 of 36 messages. Exact phrase retention is not semantic correctness, factual accuracy or actionability. Fully extractive output is primarily a construction check: copying can preserve a false user statement or omit a critical negation or qualifying sentence.

Review challenge results

Authored challenge group	Flagged	Missed or mismatched
Off-domain requests	7 / 8	1 not flagged
Ambiguous or multiple-issue requests	5 / 8	3 not flagged
Straightforward in-domain requests	1 / 8	1 flagged against expected no-review

The ambiguous group mixes four insufficient-context cases and four clearly multi-intent cases. The 5-of-8 result is for that combined group, not a pure multi-issue recall estimate. The one straightforward mismatch is a lost-card request: the authored expectation was no review, while the fixed security rule escalates it. That mismatch reflects a policy disagreement as well as a diagnostic result.

How to interpret this evidence

These fixtures were authored by an AI assistant for the project, not sampled from customer records. They are small, intentionally selected tests with lexical traps and difficult issue placement. No model, threshold, extractor or review rule was tuned after their scores were inspected. A new independent set is needed if development responds to these failures.

Detailed outputs and missing spans are retained in reports/summary_diagnostics.csv and reports/review_diagnostics.csv; authoring methods and limitations are in data/FIXTURES.md.

07 CASE STUDY

Failure analysis and next validation

The most useful next step is independent validation on an authorized target workflow, not a broader claim based on the current benchmark.

A confident off-domain failure

The synthetic input “My graphics card is not working after a computer restart. The screen stays black.” is assigned card_not_working with 86.78% intent confidence. No review rule fires. The word “card” satisfies the banking cue check, and the closed-set classifier cannot represent a separate out-of-domain class.

This is an observed exploratory failure, not robust out-of-domain detection. Similar lexical traps can look familiar to the model while referring to a different task. The example remains in the diagnostic record; rules were not patched to make the demonstration look stronger.

Confident errors inside the benchmark

Official test example	Expected and predicted intent	Lesson
“can I have a non virtual card?”	order_physical_card → getting_virtual_card 97.26% confidence; no flag	Negation can reverse the intended request
“My transfer is pending.”	balance_not_updated_after_bank_transfer → pending_transfer 93.55% confidence; no flag	Short text can cross queue boundaries
“Where do I find the exchange rate?”	exchange_charge → exchange_rate 95.12% confidence; no flag	Benchmark labels can also be hard to infer from surface text

These are disagreements with benchmark labels, not adjudicated service errors. The exchange example especially warrants label review before treating it as model misunderstanding. Other frequent confusions involve card disputes, failed versus reverted top-ups and transfer status. The full error log keeps source text, both labels, confidence and review reasons.

A staged validation plan

1. Establish the intended domain. Obtain permission to use de-identified target-domain messages. Define queue ownership, multi-issue handling and meaningful error costs with support staff. Create new labels through independent review and adjudication.

2. Evaluate on a fresh holdout. Use time- or source-separated data with stronger paraphrase audits. Measure per-intent and per-queue performance, rejection behavior, false escalations, missed security concerns, multi-intent cases and off-domain traps.

3. Test usefulness with people. Have reviewers assess summary completeness, negation, timing and actionability. Compare against simpler interfaces in a controlled study before estimating time savings or quality improvements.

4. Add deployment controls. Require privacy and security review, access controls, retention rules, auditability, monitoring and human escalation before any live integration. Keep account and financial actions outside this prototype.

08 CASE STUDY

Reproduction and source references

The project includes executable source, a notebook, a local demo, a saved model and row-level evaluation artifacts. No paid model API or external model credential is required.

Run the local demonstration

Use Python 3.12 in a fresh virtual environment. From the project folder, install the pinned runtime dependencies and launch Streamlit. The bundled model is compatible with the recorded dependency versions; retrain if loading fails.

```
python -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
streamlit run app.py
```

Use only public or synthetic messages. Load only a trusted joblib model: deserialization can execute code. Do not expose the demo publicly without a deployment and privacy review.

Reproduce the evidence

```
python -m pip install -r requirements-dev.txt
python -m src.train
python -m src.evaluate_workflow
python -m pytest -q
```

The recorded run uses Python 3.12.14, scikit-learn 1.8.0 and seed 42. All 26 software tests pass. The executed notebook, per-class results, threshold curves, split audit, bootstrap intervals and diagnostic CSVs support review of the results. Exact runtimes can vary by machine; software tests do not establish production performance.

Evidence map

Classifier and calibration results: reports/metrics.json. Threshold and full-policy behavior: reports/operating_policy.json. Fixture diagnostics: reports/workflow_metrics.json. Dataset hygiene: reports/split_audit.json. Row-level failures: reports/error_analysis.csv. Reproducible analysis: notebooks/01_support_ticket_triage.ipynb.

Sources and licensing

[1] PolyAI. BANKING77, task-specific-datasets repository. Pinned commit 57ec275d8078af65b7731c2a98be812d844a6d6b; retrieved 18 October 2026. https://github.com/PolyAI-LDN/task-specific-datasets

[2] Casanueva, I., Temcinas, T., Gerz, D., Henderson, M. and Vulić, I. (2020). Efficient Intent Detection with Dual Sentence Encoders. 2nd Workshop on NLP for ConvAI. https://arxiv.org/abs/2003.04807

[3] PolyAI. BANKING77 dataset card. https://huggingface.co/datasets/PolyAI/banking77

BANKING77 and its accompanying source materials retain the Creative Commons Attribution 4.0 International license: https://creativecommons.org/licenses/by/4.0/. Raw files are unmodified. Processed splits, the nine-queue mapping, review rules and synthetic fixtures are project additions. Attribution and checksums are preserved with the project.

Project-authored code, documentation and synthetic fixtures are MIT licensed, copyright 2026 Keisha Walker. Dataset and dependency licenses remain separate. These results describe benchmark performance and development work, not deployed business impact or independent external validation.